

Toward an AI-Native Operating System Stack: Architecture, Interfaces, and Security in the ruvnet Ecosystem

A Combined Architectural Analysis and Threat Model

Independent Research Note

March 2026

Abstract

This paper presents a unified analysis of the ruvnet repository ecosystem on GitHub, integrating three related investigations: an architectural inference that the ecosystem exhibits a layered structure analogous to an AI-native operating system; a reference interface architecture specifying the formal contracts between those layers; and an adversarial threat model examining the failure modes and security vulnerabilities that arise precisely at the inter-layer interfaces. The repositories examined are RuFlo (agent orchestration), RuVector (vector and graph memory substrate), RuView (sensor perception pipeline), and agentic-flow (runtime and deployment bridge). The central finding is that the ecosystem, while not a literal OS kernel implementation, strongly suggests a coherent platform stack—and that realizing this vision requires not only formal interface definitions but also systematic robustness engineering across perception-to-memory and memory-to-orchestration boundaries. We conclude with a research agenda prioritizing inter-layer security, consistency guarantees, and perceptual adversarial resilience.

Contents

I Architectural Inference: The ruvnet Ecosystem as a Layered Stack	2
1 Introduction and Thesis	2
1.1 Why This Reading Is Non-Trivial	2
2 Why the Stack Interpretation Is Plausible	2
3 Layer-by-Layer Architecture	3
4 Repository-to-OS-Analogue Mapping	3

5 Evidence by Layer	4
5.1 Profile-Level Platform Framing	4
5.2 Ruflo as Orchestration Layer	4
5.3 RuVector as Memory and State Substrate	4
5.4 RuView as Perception and Sensor Ingestion	4
5.5 Agentic-Flow as Runtime and Deployment Bridge	4
6 What the Combined Pattern Suggests	5
7 Caveats and Limits of the Inference	5
II Interface Architecture: Formalizing the Stack Contracts	5
8 Introduction	5
9 Full Platform Architecture	6
10 Operating System Analogy Table	6
11 Observation Schema	6
12 Memory Model	7
13 Memory API	7
14 Capability Interface	8
15 Agent Scheduling and Task Specification	8
16 Consistency Model	9
17 Security Model	9
18 Observability Requirements	9
19 Example Execution Flow	10
20 MVP Integration Roadmap	10
III Threat Model: Failure Modes and Security Vulnerabilities	10
21 Introduction: Why a Threat Model Is Necessary	10
22 Threat Taxonomy	11
23 Perception-Layer Threat Analysis	11
23.1 The Fundamental Problem: Physics-to-Embedding Impedance	11
23.2 Threat P-1: Adversarial CSI Injection	11
23.3 Threat P-3: Confidence Inflation	11

24 Memory-Layer Threat Analysis	12
24.1 Threat M-1: Memory Poisoning	12
24.2 Threat M-2: Embedding Manipulation	12
24.3 Threat M-5: Working Memory Leak	12
25 Orchestration-Layer Threat Analysis	12
25.1 Threat O-1: Task Injection	12
25.2 Threat O-4: Cascading Retry Storm	12
26 Capability-Layer Threat Analysis	13
26.1 Threat C-2: Confused Deputy	13
26.2 Threat C-4: Side-Effect Laundering	13
27 Cross-Layer Attack Scenarios	13
27.1 Scenario A: Perception-Poisoning to Orchestration	13
27.2 Scenario B: Memory Poisoning to Capability Escalation	14
28 Robustness Requirements	14
29 Security Research Agenda	15
 IV Conclusion and Unified Research Agenda	 15
30 Summary of Findings	15
31 The Critical Insight: Interfaces as Both Enabler and Attack Surface	16
32 Recommended Next Steps	16
33 Final Assessment	16

Part I

Architectural Inference: The ruvnet Ecosystem as a Layered Stack

1 Introduction and Thesis

Most public GitHub profiles present collections of loosely related experiments. The ruvnet profile [1] is different. Taken together, its principal repositories suggest a layered architecture organized around persistent AI agents: perception and ingestion, memory and state, orchestration and execution, deployment and runtime, and supporting ecosystem services. The pattern is strong enough to merit a systems interpretation.

The central claim of this first part is not that the ruvnet repositories literally implement an operating system kernel. It is that several repositories, when viewed together, exhibit a layered architecture analogous to an operating-system or distributed-systems stack. The evidence is drawn from the public GitHub profile and repository descriptions for Ruflo [2], RuVector [3], RuView [4], and agentic-flow [5].

The conclusion is deliberately cautious: the “AI-native operating system stack” interpretation is a well-supported architectural inference, not a proven statement of maintainer intent.

1.1 Why This Reading Is Non-Trivial

The strongest evidence comes from the profile-level framing and from how the major repositories describe themselves. The ruvnet GitHub profile explicitly presents the account as containing “Core frameworks and libraries for building AI-powered applications and intelligent systems” [1]. The profile groups projects into categories such as AI frameworks, agentic systems, automation, and security-oriented tooling—that is platform language, not merely application language.

2 Why the Stack Interpretation Is Plausible

A classical operating system provides abstractions for processes, memory, files, devices, networking, security, and user interaction. An AI-native stack, by analogy, would replace or supplement those abstractions with agent-oriented ones: agents or swarms instead of ordinary processes, vector or graph memory instead of plain files alone, sensor and stream ingestion instead of conventional device drivers, tool registries and agent naming instead of minimal RPC, and safety or verification services tuned to autonomous workflows.

This is not a literal one-to-one mapping, and it should not be overstated. Still, the public material points in that direction. Most notably, the RuVector README [3] includes a section titled “A Complete Agentic AI Operating System” and states that RuVector is not merely a database one adds to a stack, but rather the entire stack. That wording is a public architectural claim in the repository itself. Even if one discounts the marketing flourish, it directly supports reading the ecosystem in stack terms.

3 Layer-by-Layer Architecture

The inferred architecture is organized as five principal layers, shown in Figure 1.

4 Repository-to-OS-Analogue Mapping

Table 1 presents the principal repository mappings to classical OS analogues.

Repository	Described Role	OS Analogue	Why the Analogy Fits
RuFlo [2]	Agent orchestration platform for intelligent multi-agent swarms and autonomous workflows	Scheduler / process supervisor / control plane	Coordinates tasks, swarms, and execution flow, much as an OS scheduler and supervisor coordinate processes
RuVector [3]	High-performance, real-time, self-learning vector/-graph database in Rust; described as “the entire stack”	Memory manager + filesystem + retrieval substrate	Stores and organizes state, embeddings, graphs, and queryable memory, analogous to system memory plus persistent data abstractions
RuView [4]	WiFi-based sensing that turns CSI into pose estimation, vital signs, and presence detection	Device driver / sensor stack / perception layer	Converts noisy physical-world signals into structured machine-usable state, analogous to device and sensor subsystems
agentic-flow [5]	Deployment layer for switching models and turning Claude-code-style agent work into hosted cloud agents	Runtime / shell / deployment environment	Bridge from local agent development to deployed runtime environments, analogous to shell plus runtime manager
ANS, Federated MCP [1]	Agent discovery, registries, distributed AI services, tool integration	Naming / networking / service discovery / middleware	Resemble network and middleware services that let autonomous components find and use each other

Table 1: Repository-to-OS-analogue mapping for the ruvnet ecosystem.

5 Evidence by Layer

5.1 Profile-Level Platform Framing

The profile README [1] is the broadest piece of evidence. It does not present the repositories as isolated experiments. Instead it describes them as a collection of core frameworks and libraries for building AI-powered applications and intelligent systems, then enumerates projects for orchestration, discovery, search, MCP integration, streaming, security, and WiFi sensing. That is exactly the kind of breadth one expects from someone thinking in platform layers.

5.2 Ruflo as Orchestration Layer

The Ruflo repository [2] presents itself as an agent orchestration platform for deploying intelligent multi-agent swarms, coordinating autonomous workflows, and building conversational AI systems. In OS language, this makes Ruflo the best candidate for the scheduler and supervisor layer: the component that decides which agents do what, in which order, under which workflow constraints.

5.3 RuVector as Memory and State Substrate

The RuVector repository [3] provides the clearest explicit support for the stack reading. It describes itself as a high-performance, real-time, self-learning vector graph neural network and database built in Rust. More importantly, it includes a section titled “A Complete Agentic AI Operating System” and claims to include everything an AI application needs to run from bare metal hardware up to the application layer. It also advertises search, unified storage, a PostgreSQL drop-in replacement, and built-in graph capabilities including Cypher and SPARQL. Those claims may be ambitious, but architecturally they are exactly what one would expect from a lower-layer substrate intended to hold agent memory, graph structure, retrieval state, and queryable context.

5.4 RuView as Perception and Sensor Ingestion

RuView [4] is important because it broadens the ecosystem beyond text and orchestration. The repository describes turning commodity WiFi signals into real-time human pose estimation, vital-sign monitoring, and presence detection without video, by analyzing channel-state information (CSI) disturbances with physics-based signal processing and machine learning. It explicitly notes that full pose estimation and vital-sign monitoring require CSI-capable hardware, and that ordinary consumer hardware is much more limited.

That caveat matters and makes the reading more careful. Whether or not all headline sensing claims generalize in practice, the role of the repository within the broader ecosystem is clear: it acts as a perception layer that converts messy physical inputs into structured state.

5.5 Agentic-Flow as Runtime and Deployment Bridge

The agentic-flow repository [5] describes letting users switch between lower-cost AI models in Claude Code or the Agent SDK, and then deploy fully hosted agents for real business

purposes in their preferred cloud. This makes agentic-flow the layer that turns a locally developed agent into an operational service—the runtime or shell analogy is the best fit.

6 What the Combined Pattern Suggests

If one steps back, the pattern is not merely that there are many repositories. It is that they are complementary. The key repositories line up along the path that an autonomous system would need:

1. Sense or ingest the world (signals, streams, browser interactions, data sources).
2. Represent and remember what was sensed (vectors, graphs, embeddings, query state).
3. Coordinate and decide through agents or swarms (tasks, routing, handoffs, work-flows).
4. Deploy and operate those agents in a real runtime environment.
5. Secure and connect them with discovery, naming, and tool/service protocols.

That is the central reason the ecosystem looks stack-like rather than accidental.

7 Caveats and Limits of the Inference

Two cautions are important.

First, repository descriptions are not independent validation. They are useful evidence of architectural intent and self-presentation, but they do not prove production maturity, broad empirical performance, or completion of all advertised features.

Second, one should distinguish between two interpretations:

Strong form: there is a deliberate long-range plan to build an AI-native platform stack.

Weak form: an experienced, AI-accelerated systems builder naturally generates adjacent repositories that only look like a stack because the problems are related.

The public GitHub evidence strongly supports the existence of a layered pattern. It does not prove which interpretation is correct.

Part II

Interface Architecture: Formalizing the Stack Contracts

8 Introduction

Most modern AI systems are implemented as isolated applications built around large language models and ad-hoc workflows. However, persistent autonomous systems require

a more structured runtime environment. An AI-native operating system platform would provide persistent semantic memory, structured perception inputs, agent orchestration and scheduling, secure capability invocation, and system-wide observability.

The architectural inference in Part I identified the layers. This part specifies what is needed to make those layers function as a coherent system: formal interface definitions, canonical data schemas, and inter-layer protocols.

9 Full Platform Architecture

The complete platform architecture with explicit inter-layer interfaces is:

```

Applications / Operator Interfaces
|
Workflow Policies / Agent Programs
|
  Ruflo Orchestration Layer
|
  Capability Registry + Security Policies
|
  RuVector Memory Substrate
|
    Event Bus / State Log
|
  RuView Perception Adapters
|
Sensors / Hardware / Model Runtimes

```

Each layer communicates through formal, versioned interfaces. The critical insight is that the OS analogy only becomes architecturally serious once the contracts between layers are specified rather than implied.

10 Operating System Analogy Table

Table 2 extends the analogy mapping from Part I with the interface-level correspondences.

11 Observation Schema

All perception layers must emit standardized observation objects. The canonical schema is:

```

Observation {
  source          : string,
  timestamp       : datetime,
  entity_id       : string,
  observation_type: string,
  payload         : object,
  confidence      : float,
  provenance      : object,
}

```

AI Platform Layer	Classical OS Analogue
Applications	User-space programs
Workflow definitions	Shell scripting
Ruflo orchestration	Process scheduler
Capability registry	System call table
Security policies	Kernel security layer
RuVector memory	Filesystem + virtual memory
Event bus	Interprocess communication (IPC)
RuView adapters	Device drivers
Model runtimes	CPU/GPU runtime

Table 2: Extended OS analogy with interface-layer correspondences.

```

retention_policy: string,
access_policy    : string
}

```

The inclusion of `confidence`, `provenance`, and `retention_policy` as first-class fields reflects genuine systems thinking. These are not afterthoughts: confidence scores propagate through downstream reasoning, provenance enables auditability, and retention policies enforce data lifecycle governance across the memory substrate.

12 Memory Model

The memory subsystem supports four categories, grounded in cognitive architecture research:

Episodic memory Time-indexed records of past observations and events.

Semantic memory Factual knowledge as entity-relation-value triples with provenance.

Structural graph memory Graph-structured relational knowledge queryable via Cypher or SPARQL.

Working task memory Transient state scoped to an active task, committed or discarded on completion.

13 Memory API

The canonical memory API exposes the following operations:

```

put_episode(event)
put_fact(entity, relation, value, provenance)
put_embedding(object_id, vector, metadata)
query_graph(pattern)
query_semantic(query, scope, k)
open_working_set(task_id)
commit_task_state(task_id, diff)

```

All write operations must include provenance metadata. Read operations return confidence-weighted results where applicable.

14 Capability Interface

Capabilities represent the basic operations agents may invoke. The capability descriptor schema is:

```

Capability {
  name           : string,
  input_schema   : JSONSchema,
  output_schema  : JSONSchema,
  latency_class  : enum {realtime, interactive, batch},
  side_effect_class : enum {pure, local, external,
    destructive},
  required_permissions: list<Permission>,
  audit_level      : enum {none, log, attest}
}

```

Example capabilities include `perception.observe`, `memory.semantic_query`, `graph.update`, `browser.navigate`, and `workflow.spawn_agent`. The `side_effect_class` field is architecturally important: it allows the orchestration layer to reason about reversibility before scheduling destructive operations.

15 Agent Scheduling and Task Specification

Rufo coordinates autonomous agents via a formal task protocol. Responsibilities include task scheduling, workflow routing, parallel execution, retry policies, and escalation. The canonical task specification is:

```

Task {
  task_id           : uuid,
  parent_task_id    : uuid | null,
  goal              : string,
  constraints        : list<Constraint>,
  resource_budget    : ResourceBudget,
  required_capabilities: list<CapabilityRef>,
  memory_scope       : MemoryScopeRef,
  priority           : int,
  deadline           : datetime | null,
  termination_condition: TerminationSpec
}

```

```
}

```

The `parent_task_id` field supports hierarchical task decomposition. The `memory_scope` field links each task to an isolated working memory namespace, preventing cross-task state contamination.

16 Consistency Model

Three forms of system state exist, with different consistency guarantees:

Streaming observations Append-only, at-least-once delivery, no global ordering guarantee.

Committed facts Durable, linearizable within an entity scope, versioned.

Derived agent conclusions Soft-state, confidence-weighted, subject to revision.

All mutations must include provenance metadata. Conflicts between derived conclusions and newly committed facts must be resolved by explicit reconciliation policy rather than silent overwrite.

17 Security Model

All actors are principals: humans, agents, services, sensors, and runtimes. Authorization follows a capability-based model:

```
principal X
  may invoke capability Y
  on resource Z
  within scope S
  until expiration T
  with attenuation A
```

Capability-based security is the appropriate model because it naturally supports least-privilege agent operation and attenuation across delegation chains. Key requirements include: capabilities must be unforgeable tokens; delegation must be explicitly attenuated (a sub-agent cannot receive more authority than its parent); revocation must be propagatable across active delegation chains; and sensor-originated capabilities must be treated as lower-trust than operator-originated capabilities.

18 Observability Requirements

System telemetry must capture, for every operation: caller identity, invoked capability, parameters (sanitized), memory access patterns, state mutations, latency, and confidence metrics. This telemetry is the foundation for the forensic analysis required after inter-layer failures—a theme developed at length in Part III.

19 Example Execution Flow

A canonical end-to-end execution traverses all layers:

1. A sensor event is emitted by RuView and tagged with the observation schema.
2. The event is stored in RuVector episodic memory via `put_episode`.
3. The Ruflo scheduler detects a trigger condition on the event stream.
4. An agent opens a working memory set and queries semantic memory.
5. The agent invokes a capability, subject to security policy check.
6. The result is committed to structural memory with provenance.
7. Working memory is committed or discarded, closing the task scope.

20 MVP Integration Roadmap

Stage 1: Schema Definition Define observation schema, memory APIs, and capability descriptors; build serialization libraries and schema validators.

Stage 2: Memory Layer Integrate RuVector storage; enable semantic search and graph query; implement provenance tracking.

Stage 3: Orchestration Connect Ruflo scheduler; implement task protocol; build retry and escalation logic.

Stage 4: Perception Normalize RuView outputs to observation schema; connect event streams; implement confidence propagation.

Stage 5: Security and Observability Deploy identity and capability-minting system; add telemetry tracing; implement revocation.

Part III

Threat Model: Failure Modes and Security Vulnerabilities

21 Introduction: Why a Threat Model Is Necessary

The interface architecture in Part II specifies what the stack should do when everything works correctly. This part asks the harder question: what happens when it does not?

In agentic systems, the most dangerous failure modes run precisely along the inter-layer interfaces. A corrupted observation propagating from RuView through RuVector into a Ruflo-scheduled agent action can produce consequential real-world effects before any human observer detects the anomaly. The threat model developed here is organized around three principal attack surfaces: the perception-to-memory boundary, the memory-to-orchestration boundary, and the orchestration-to-capability boundary.

22 Threat Taxonomy

We organize threats into four classes:

Class P: Perception-layer attacks Manipulation or corruption of sensor data before or during ingestion.

Class M: Memory-layer attacks Corruption, poisoning, or unauthorized access to the memory substrate.

Class O: Orchestration-layer attacks Manipulation of task scheduling, workflow routing, or agent coordination.

Class C: Capability-layer attacks Unauthorized capability invocation, privilege escalation, or confused deputy attacks.

Table 3 summarizes the principal threats in each class.

23 Perception-Layer Threat Analysis

23.1 The Fundamental Problem: Physics-to-Embedding Impedance

The most underappreciated vulnerability in the stack is the impedance mismatch between RuView’s physics-domain outputs (CSI signal matrices, confidence intervals over physical poses) and RuVector’s embedding-space representations. Bridging these two representations faithfully is non-trivial, and the conversion process is a natural attack surface.

A CSI signal can be adversarially crafted to produce arbitrary outputs from a neural network perception pipeline, in a manner exactly analogous to adversarial examples in image classification. Unlike image classification failures, however, a false pose or presence detection feeds directly into persistent episodic memory—the corruption is not transient.

23.2 Threat P-1: Adversarial CSI Injection

An adversary with physical access to the RF environment can craft WiFi signals designed to fool the CSI analysis pipeline into producing specific false outputs. Mitigations include: multi-source sensor fusion requiring agreement across independent physical modalities before committing observations; anomaly detection on CSI signal statistics prior to neural processing; and explicit “observation quality” flags that prevent low-quality perceptions from triggering high-stakes orchestration actions.

23.3 Threat P-3: Confidence Inflation

The observation schema includes a **confidence** field. If the perception pipeline can be manipulated to inflate confidence scores, downstream components that gate actions on confidence thresholds will be bypassed. The mitigation is to treat confidence as an upper-bounded estimate computed from multiple independent signals, not as a single pipeline output that can be freely set.

24 Memory-Layer Threat Analysis

24.1 Threat M-1: Memory Poisoning

An agent with write access to semantic memory can inject false facts. Unlike in traditional databases, where facts are relatively easy to verify against external ground truth, AI agent memory may contain soft assertions (beliefs, inferences, plan states) for which ground truth is unavailable. This makes memory poisoning in agentic systems qualitatively more dangerous than SQL injection in relational databases.

Mitigations include: write-access differentiation (separating observation-derived facts from agent-derived conclusions, with different trust levels); mandatory provenance on all writes; and periodic memory consistency audits against high-confidence external sources.

24.2 Threat M-2: Embedding Manipulation

Vector retrieval systems are vulnerable to adversarial embedding injection: an attacker inserts vectors positioned in the high-dimensional space to be retrieved preferentially in response to legitimate queries. The memory API's `query_semantic` operation is the primary attack surface. Mitigations include embedding-space anomaly detection, diversity enforcement in retrieval results, and access control on the `put_embedding` operation.

24.3 Threat M-5: Working Memory Leak

The task specification includes a `memory_scope` field intended to isolate working memory per task. If scope isolation is not enforced at the RuVector layer—not merely as a naming convention but as an access control primitive—then a compromised agent can read or write across task boundaries, violating the confidentiality of parallel executing agents.

25 Orchestration-Layer Threat Analysis

25.1 Threat O-1: Task Injection

If the Rufflo task submission interface is not properly authenticated, an adversary can inject arbitrary tasks into the scheduler. This is analogous to process injection in traditional OS security. The mitigation requires that all task submissions be signed by an authorized principal, and that the scheduler verify the signature before enqueueing.

25.2 Threat O-4: Cascading Retry Storm

The task specification includes retry policies. An adversary can craft a task designed to fail predictably, triggering exponential retry behavior that consumes orchestration resources. Mitigation requires circuit-breaker logic: retry budgets must be global, not per-task, with backoff enforced at the scheduler level rather than delegated to individual tasks.

26 Capability-Layer Threat Analysis

26.1 Threat C-2: Confused Deputy

The confused deputy attack is the most subtle threat in an agentic capability system. A legitimate agent with broad permissions is instructed (via crafted memory content, injected task parameters, or adversarial prompt) to invoke a capability on behalf of an attacker who lacks the required permissions directly. The agent acts in good faith but produces attacker-desired effects.

Mitigations include: capability invocation always recording the full provenance chain (original principal, delegating agent, invoking agent); the capability runtime checking not only whether the invoking agent has permission but whether the workflow context is consistent with the requested action; and human-in-the-loop requirements for capabilities with `destructive` side-effect class.

26.2 Threat C-4: Side-Effect Laundering

If capability classification is coarse or incorrect, an attacker can use a capability classified as `pure` or `local` to trigger external effects (e.g., a memory write that is read by an external monitoring system). The mitigation is formal verification of capability side-effect classes, not self-declaration by capability implementers.

27 Cross-Layer Attack Scenarios

The most dangerous attacks are not single-layer but cross-layer chains. Two canonical scenarios illustrate the risk.

27.1 Scenario A: Perception-Poisoning to Orchestration

1. Adversary crafts RF signals causing RuView to falsely report a person present in a restricted area (P-1).
2. False observation committed to RuVector episodic memory with high confidence (P-3).
3. Rufflo scheduler detects trigger condition and spawns a security-response agent (O-1 consequence).
4. Agent invokes a physical access control capability based on false perception state (C-2).
5. Real-world physical access control system takes action on false premises.

This chain traverses all four layers. No single-layer defense is sufficient; the mitigation requires cross-layer provenance tracking and confidence gating at the orchestration layer.

27.2 Scenario B: Memory Poisoning to Capability Escalation

1. Attacker with limited memory write access injects a false semantic fact asserting elevated permissions for a target agent (M-1, M-3).
2. The target agent queries its permission set from semantic memory and receives the forged result.
3. The agent invokes a capability beyond its actual authorization (C-1).
4. Orchestration layer, relying on memory-derived permission checks, does not detect the escalation.

Mitigation requires that capability permissions be stored in a tamper-evident registry separate from general semantic memory, and that the capability runtime verify permissions against the registry directly rather than trusting agent-provided claims.

28 Robustness Requirements

Synthesizing the threat analysis, we derive the following cross-cutting robustness requirements for the ruvnet stack:

- R1. Provenance transitivity.** Every fact, observation, and conclusion must carry a full provenance chain traceable to its physical or human source. Provenance must be cryptographically signed and immutable.
- R2. Confidence gating.** The orchestration layer must enforce minimum confidence thresholds for high-stakes capability invocations, and those thresholds must be set by policy rather than by individual agents.
- R3. Cross-layer forensics.** Observability infrastructure must support reconstruction of the full causal chain for any agent action, spanning all layers from sensor event to capability invocation.
- R4. Memory-permission separation.** Capability authorization must be stored in a tamper-evident registry isolated from general semantic memory; agents must not be able to modify their own permission records.
- R5. Human-in-the-loop for destructive actions.** Capabilities with destructive side-effect class must require human attestation unless explicitly waived by an authorized policy.
- R6. Sensor diversity.** High-stakes orchestration triggers must require corroboration from at least two independent physical sensing modalities before committing to memory.
- R7. Scope isolation enforcement.** Working memory scopes must be enforced as access-control primitives at the memory substrate layer, not merely as naming conventions.
- R8. Global retry budgets.** Retry policies must consume from a global budget managed by the orchestration layer, with circuit-breakers enforced system-wide.

29 Security Research Agenda

The threat model identifies several open research problems that the ruvnet ecosystem, and the AI-native OS field broadly, must address:

Adversarial robustness of CSI-based perception The existing literature on adversarial examples in image classification must be extended to WiFi CSI signals and pose estimation pipelines. The physics of RF propagation create both constraints (limited signal degrees of freedom) and opportunities (physical-layer anomaly detection) not present in image domains.

Embedding-space integrity Formal definitions of integrity for vector memory stores are largely absent from the current literature. The analogy to database integrity constraints is productive but incomplete; vector spaces require geometric integrity notions not captured by relational consistency conditions.

Agentic confused deputy formalization The confused deputy problem has a rich formal literature in OS security. Extending this formalization to multi-agent systems with natural-language goal specifications, where the “instruction” triggering misuse may be embedded in retrieved memory rather than explicit API parameters, is an open problem.

Cross-layer causal audit Distributed tracing systems (e.g., OpenTelemetry) provide the tooling substrate, but the semantic layer—linking a physical sensor event to a downstream capability invocation in a human-auditable way—requires new work on causal graph construction across heterogeneous layer types.

Part IV

Conclusion and Unified Research Agenda

30 Summary of Findings

This paper has presented three integrated analyses of the ruvnet ecosystem:

Part I established that the ecosystem exhibits a layered architecture analogous to an AI-native operating system, with RuFlo, RuVector, RuView, and agentic-flow playing roles analogous to scheduler, memory/filesystem, device drivers, and runtime shell respectively. The inference is well-supported by public repository descriptions but should be understood as an architectural analogy rather than a literal kernel claim.

Part II specified the formal interface architecture required to realize the inferred stack as a coherent system: canonical observation schemas, a four-category memory model with explicit API, a capability descriptor format with side-effect classification, a task specification protocol, a consistency model distinguishing streaming, committed, and derived state, and a capability-based security model with delegation and revocation.

Part III developed a threat model organized around four attack classes (perception, memory, orchestration, capability) and derived eight cross-cutting robustness requirements.

The most dangerous threats are cross-layer attack chains that exploit the interfaces specified in Part II. The security model in Part II is necessary but not sufficient; it must be supplemented by the robustness requirements derived from the threat analysis.

31 The Critical Insight: Interfaces as Both Enabler and Attack Surface

The central insight connecting all three parts is that the same inter-layer interfaces that make the ruvnet ecosystem function as a coherent stack also constitute its primary attack surface. The observation schema is necessary for interoperability and dangerous if confidence fields can be inflated. The memory API is necessary for agent cognition and dangerous if write access is not properly differentiated. The capability interface is necessary for controlled action and dangerous if side-effect classes are self-declared rather than verified.

This is not a reason to abandon the interface architecture. It is a reason to treat interface security as a first-class engineering concern rather than a post-hoc addition.

32 Recommended Next Steps

1. Implement the observation schema and memory API as specified, with cryptographic provenance from the outset.
2. Develop a formal capability registry with tamper-evident storage and explicit attenuation semantics.
3. Build a cross-layer observability system capable of reconstructing full causal chains from sensor event to capability invocation.
4. Commission adversarial testing of the RuView CSI pipeline against crafted RF signals before connecting it to production memory or orchestration systems.
5. Establish a working memory scope isolation primitive at the RuVector layer and verify its enforcement through formal methods or systematic adversarial testing.
6. Engage the broader AI safety and OS security research communities around the open problems identified in Section 22.

33 Final Assessment

The ruvnet ecosystem represents one of the most coherent public attempts to instantiate an AI-native operating system stack. The complementarity of its components is not accidental in appearance, and the interface architecture required to realize the vision is well within the current state of the art. The threat model developed here is not an argument against the project; it is a map of the engineering work that must accompany the architectural vision if the resulting system is to be deployed safely in consequential environments.

An AI-native OS that is architecturally elegant but perceptually manipulable, or that has rigorous memory APIs but no inter-layer provenance, is not a platform—it is an attack surface wearing a platform’s clothes. The research agenda in Section 22 outlines the path from the current state to a system that is both.

References

- [1] ruvnet GitHub Profile. *Core frameworks and libraries for building AI-powered applications and intelligent systems*. <https://github.com/ruvnet> Accessed March 2026.
- [2] ruvnet. *Ruflo: Agent Orchestration Platform for Claude*. GitHub Repository. <https://github.com/ruvnet/ruflo> Accessed March 2026.
- [3] ruvnet. *RuVector: High-Performance Real-Time Self-Learning Vector Graph Neural Network and Database*. GitHub Repository. <https://github.com/ruvnet/ruvector> Accessed March 2026.
- [4] ruvnet. *RuView: WiFi-Based Human Sensing and Perception Pipeline*. GitHub Repository. <https://github.com/ruvnet/RuView> Accessed March 2026.
- [5] ruvnet. *Agentic-Flow: Deployment and Runtime Bridge for Hosted AI Agents*. GitHub Repository. <https://github.com/ruvnet/agentic-flow> Accessed March 2026.
- [6] Dennis, J.B. and Van Horn, E.C. *Programming Semantics for Multiprogrammed Computations*. Communications of the ACM, 9(3):143–155, 1966.
- [7] Saltzer, J.H. and Schroeder, M.D. *The Protection of Information in Computer Systems*. Proceedings of the IEEE, 63(9):1278–1308, 1975.
- [8] Goodfellow, I., Shlens, J., and Szegedy, C. *Explaining and Harnessing Adversarial Examples*. International Conference on Learning Representations (ICLR), 2015.
- [9] Anderson, C.R. and Rappaport, T.S. *In-building wideband partition loss measurements at 2.5 and 60 GHz*. IEEE Transactions on Wireless Communications, 3(3):922–928, 2004.

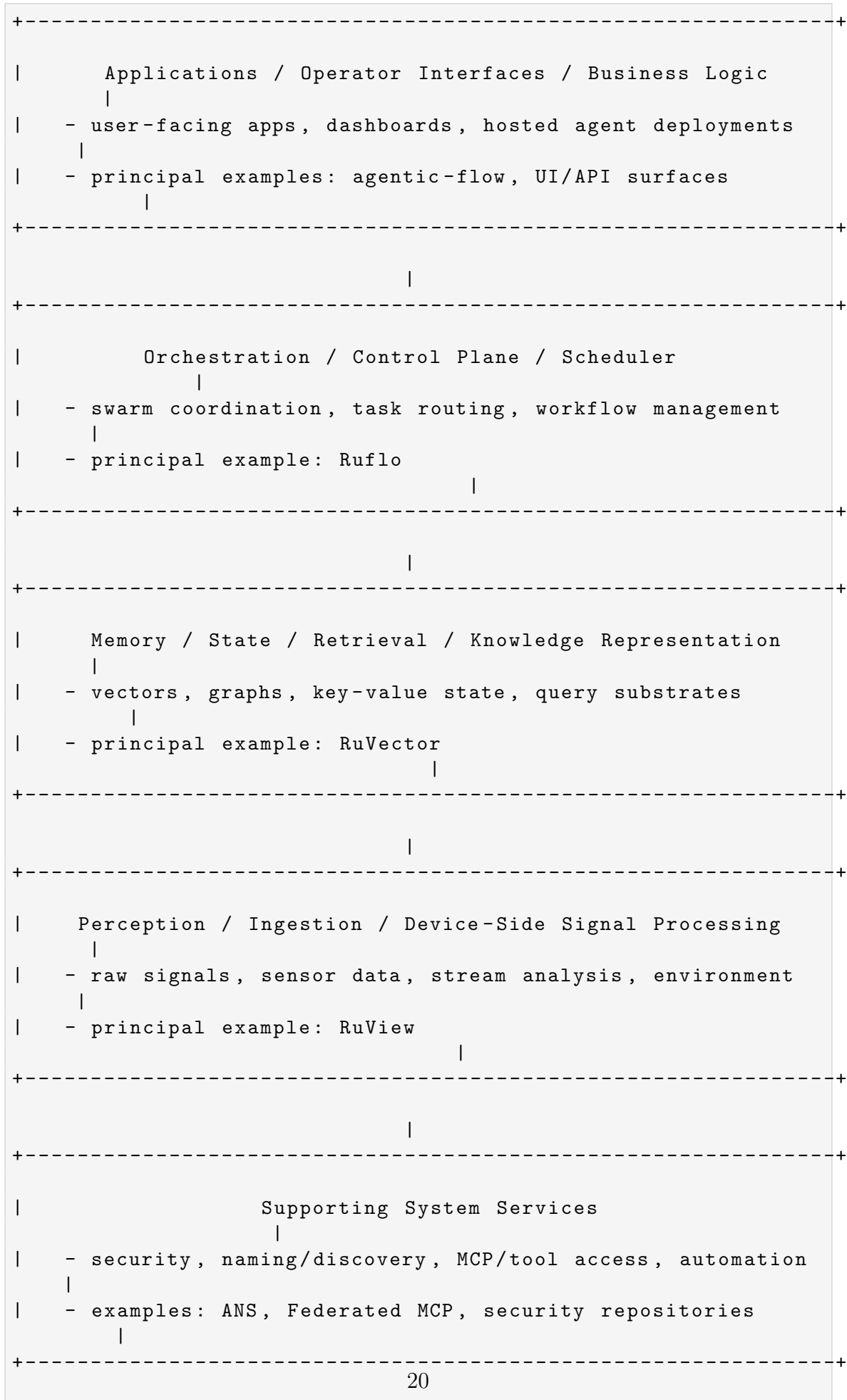


Figure 1: Inferred AI-native OS stack for the ruvnet ecosystem. This diagram represents

Class	Threat	Attack Vector	Consequence
P-1	Adversarial CSI injection	Malicious RF signal crafted to fool WiFi sensing	False presence/pose detection propagated to memory
P-2	Sensor spoofing	Physical-layer replay of captured CSI frames	Stale or fabricated environmental state
P-3	Confidence inflation	Manipulation of signal processing pipeline	Low-quality observations tagged as high-confidence
P-4	Observation flooding	High-rate event injection	Event bus saturation, memory exhaustion
M-1	Memory poisoning	Injection of false facts into semantic memory	Persistent corruption of agent reasoning base
M-2	Embedding manipulation	Crafted vectors placed near high-value query regions	Retrieval of attacker-controlled content
M-3	Provenance forgery	Forged provenance metadata on committed facts	False attribution of corrupted facts
M-4	Graph structure attack	Unauthorized insertion of misleading graph edges	Corrupted relational reasoning
M-5	Working memory leak	Cross-task memory scope violation	Information disclosure between isolated agents
O-1	Task injection	Unauthorized task submission to Ruflo scheduler	Execution of attacker-specified agent actions
O-2	Workflow hijacking	Modification of task graph during execution	Redirection of agent actions mid-workflow
O-3	Priority inversion	Manipulation of task priority fields	Denial of service to legitimate high-priority tasks
O-4	Cascading retry storm	Deliberate triggering of retry loops	Resource exhaustion across orchestration layer
C-1	Capability escalation	Agent claims permissions not granted by parent	Unauthorized external actions
C-2	Confused deputy	Legitimate agent tricked into invoking	Privilege abuse without direct escalation